Gym Management System: Testing Strategy

Software Engineering team project · Sawyer Hess, Savvas Christou, Elif Akcivi, Chinguun · October 23, 2026
Repository: github.com/savvaschristou/gym-management-system (branch main)

Summary

Requirement	Result	Where
Test plan	Scope, four testing levels, tools, entry and exit criteria, environment	Test Plan section
At least 15 unit tests: normal, edge and error cases	42 unit tests in 8 files, all passing	Unit Tests section
At least 70% coverage of tested modules	Every tested module at 89% statements or higher; 86.4% statements overall	Coverage Report section
At least 5 integration tests	6 integration tests across UI, service and data layers, all passing	Integration Tests section
TDD on one backlog feature	Membership status rule, built in three commits: red, green, refactor	TDD section
Coverage report and gap analysis	Report generated with Jest; two untested files explained	Coverage Report section

Run everything with `npm test -- --watchAll=false`, or `npm run test:coverage` for the coverage report. Result on October 23, 2026: 10 test suites, 48 tests, 48 passed.

Test Plan

Testing Scope

In scope for this milestone are the features built so far: logging in, signing up, staying signed in after a page refresh, role-protected pages for members and staff, the generic browser-storage repository, and the membership status rule built this week with TDD.

Out of scope: features not yet built (staff tools, lift logging, lift stats, dashboard), payments (excluded from the project), performance and load testing (a single-user browser app with no server), security testing beyond checking that passwords are stored hashed, and cross-browser testing (Chrome is the demo browser).

Testing Levels

Level	What it checks	How we do it	Owner
Unit	One class or function on its own, with its collaborators replaced by small fakes	Jest test files next to the code, e.g. SignUpService.test.ts; run on every change	The developer who writes the code
Integration	Real classes from different layers working together: UI to service, service to data	src/integration/auth.integration.test.tsx builds the app's real objects over a shared fake storage	Savvas (wiring), reviewed by all
System	The whole built app running in a browser	Production build served locally and exercised in Chrome, following the demo script below	Whole team before each merge to main
Acceptance	The features meet the user stories from the backlog	The demo script below, each step mapped to a user story, run with the team at the Friday meeting	Whole team; Sawyer records results

Tools and Frameworks

Tool	Version	Used for
Jest (through react-scripts)	27.5.1	Test runner, assertions, mocks, coverage (Istanbul)
jsdom	27.5.1	Simulated browser page for UI tests
React Testing Library	16.3.3	Rendering components and finding elements the way a user would (by label and role)
user-event	13.5.0	Simulated typing and clicking in UI tests
React Router MemoryRouter	6.30.6	Testing redirects without a real browser address bar
TypeScript compiler	4.9.5	Type checking (npx tsc --noEmit) before tests run
Chrome	Current	System and acceptance testing
Git and GitHub	n/a	Feature branches and pull requests; commit history as TDD evidence

Two browser features are missing inside Jest, so tests use small stand-ins from src/testUtils.ts: fakeStorage replaces localStorage, and fakeHasher replaces SHA-256 hashing, because Jest has no crypto.subtle.

Entry and Exit Criteria

Stage	Entry criteria (testing can start when)	Exit criteria (testing is done when)
Unit	The feature's code compiles with npx tsc --noEmit; the expected behavior is written down in the task or user story	Normal, edge and error cases are tested; all unit tests pass; the module has at least 70% statement coverage
Integration	Unit tests pass for every class involved	All integration tests pass; each new connection between layers has at least one test
System	All automated tests pass on the feature branch; npm run build succeeds	The demo script runs start to finish in Chrome with no errors in the browser console
Acceptance	System testing passed; the pull request is open for review	The team agrees each user story is met; no open critical defects; pull request approved and merged to main

Test Environment Setup

1. Install Node.js 18 or newer (tested on Node 22.22) and Git.
2. Clone the repository: `git clone https://github.com/savvaschristou/gym-management-system`
3. In the project folder, run `npm install`.
4. Unit and integration tests: `npm test -- --watchAll=false`. Plain `npm test` starts watch mode, which only runs tests for files changed since the last commit; press a to run them all.
5. Coverage: `npm run test:coverage`. The HTML report is written to coverage/lcov-report/index.html.
6. System and acceptance: `npm start`, then open http://localhost:3000 in Chrome. To start from a clean slate, clear the site's data in Chrome developer tools (Application, Storage, Clear site data).

Test data: the app saves two demo accounts on first load, member@demo.gym and staff@demo.gym, both with password demo123. Automated tests create their own data in fake storage, so they never depend on each other or on the browser.

Acceptance Test Script

Run in Chrome against the production build on October 23, 2026. All steps passed.

Step	Action	Expected result	Result
1	Open the app in a fresh browser	The login page at /login	Pass
2	Choose "Create an account" and sign up as a new member	Member home at /dashboard, showing the new name	Pass
3	Refresh the page	Still signed in on /dashboard	Pass
4	Type /admin into the address bar	Sent back to /dashboard	Pass
5	Sign out, then log in as staff@demo.gym	Staff home at /admin	Pass
6	Sign out and sign up again with the email from step 2	"An account with that email already exists"	Pass

Unit Tests

42 unit tests in 8 files. Each file covers normal cases (expected use), edge cases (boundaries and unusual but valid input) and error cases (invalid input or failures).

Test file	Tests	Normal	Edge	Error
domain/membership.test.ts (TDD)	9	3	5	1
services/auth/SignUpService.test.ts	6	2	1	3
data/LocalStorageRepository.test.ts	5	2	2	1
services/auth/AuthSession.test.ts	7	4	1	2
services/auth/LocalAuthProvider.test.ts	4	1	1	2
services/auth/DemoAuthProvider.test.ts	3	2	0	1
Login.test.tsx	3	1	0	2
ui/ProtectedRoute.test.tsx	3	1	0	2
services/auth/LoginValidator.test.ts	2	1	0	1
Total	**42**	**17**	**10**	**15**

Examples of each kind:

- Normal: SignUpService "creates a member account and signs the new user in".
- Edge: SignUpService accepts a password of exactly 8 characters and rejects 7; membershipStatus is still active on the end date itself and in grace the day after.
- Error: LocalStorageRepository "treats corrupted stored data as empty instead of crashing"; AuthSession "starts signed out when the saved session is corrupted".

As a check that the tests guard real behavior, the duplicate-email rule in SignUpService was temporarily disabled: one unit test and one integration test failed, and both passed again once the rule was restored.

Integration Tests

Six tests in src/integration/auth.integration.test.tsx. Each one builds the app's real objects (LocalStorageUserRepository, LocalAuthProvider, AuthSession, SignUpService and, in two tests, the SignUpPage form) over one shared fake storage. Only the browser's storage and password hashing are replaced.

#	Test	Components working together
1	A member who signs up can sign out and log back in with the same password	SignUpService, user repository, LocalAuthProvider, AuthSession
2	A user saved through the data layer can sign in through the service layer	LocalStorageUserRepository (data) to LocalAuthProvider and AuthSession (service)

3	An email used once is rejected later, even through a separately built app	Two complete app instances sharing one storage
4	A signed-in user is still signed in after a page refresh, until they sign out	AuthSession and storage across two app start-ups
5	Filling in the sign-up form creates the account and signs the member in	SignUpPage (UI), SignUpService, repository, AuthSession
6	The sign-up form shows the errors the service returns and saves nothing	SignUpPage (UI), SignUpService, repository

Test-Driven Development: Membership Status

Feature from the backlog (week of Oct 19, membership plans and member profiles): decide whether a membership is active, in its grace period, expired, frozen or cancelled on a given day. It became the function membershipStatus(membership, plan, today) in src/domain/membership.ts. Each stage of the Red-Green-Refactor cycle is a separate commit on main.

Commit	Stage	What changed	Tests
93f9649	Red	Added membership.test.ts with 9 tests, and membership.ts as an empty function that throws	0 of 9 pass
72f3988	Green	Simplest logic that passes; only membership.ts changed	9 of 9 pass
897592e	Refactor	Extracted a date helper and named the boundaries; only membership.ts changed	9 of 9 pass

To see the history: `git log --oneline -- src/domain/membership.ts`. The test file is untouched after the Red commit, which shows the tests came first and the code was shaped to meet them.

Red: Tests Written First

The nine tests cover the normal states, every boundary day (on the end date, the day after, the last grace day, the day after grace ends), a plan with no grace days, and an invalid date. Terminal output:

```
FAIL src/domain/membership.test.ts
    × is active before the end date
    × is frozen whatever the dates say
    × is cancelled whatever the dates say
    × is still active on the end date itself
    × is in grace the day after the end date
    × is in grace on the last grace day
    × is expired the day after the grace days run out
    × goes straight to expired when the plan has no grace days
    × throws a clear error for a date that is not YYYY-MM-DD
Tests:       9 failed, 9 total

  ● membershipStatus › is active before the end date
    Not implemented yet
    >  8 |   throw new Error('Not implemented yet');
```

Green: Simplest Code That Passes

```
export function membershipStatus(membership, plan, today) {
  if (membership.state === 'frozen') return 'frozen';
  if (membership.state === 'cancelled') return 'cancelled';

  if (!/^\d{4}-\d{2}-\d{2}$/.test(today) || isNaN(Date.parse(today))) {
    throw new Error('Invalid date: ' + today);
  }
  const todayDay = Date.parse(today) / 86400000;
  const endDay = Date.parse(membership.endDate) / 86400000;

  if (todayDay <= endDay) return 'active';
  if (todayDay <= endDay + plan.graceDays) return 'grace';
  return 'expired';
}
```

```
PASS src/domain/membership.test.ts
Tests:       9 passed, 9 total
```

Problems left on purpose: the date conversion is written twice, 86400000 is an unexplained number, and only today's date is validated, not the membership's end date.

Refactor: Same Behavior, Cleaner Code

```
const MS_PER_DAY = 24 * 60 * 60 * 1000;

function toDayNumber(isoDate) {
  const ms = Date.parse(isoDate);
  if (!/^\d{4}-\d{2}-\d{2}$/.test(isoDate) || isNaN(ms)) {
    throw new Error('Invalid date: ' + isoDate);
  }
  return ms / MS_PER_DAY;
}

export function membershipStatus(membership, plan, today) {
  if (membership.state !== 'active') return membership.state;

  const day = toDayNumber(today);
  const lastActiveDay = toDayNumber(membership.endDate);
  const lastGraceDay = lastActiveDay + plan.graceDays;

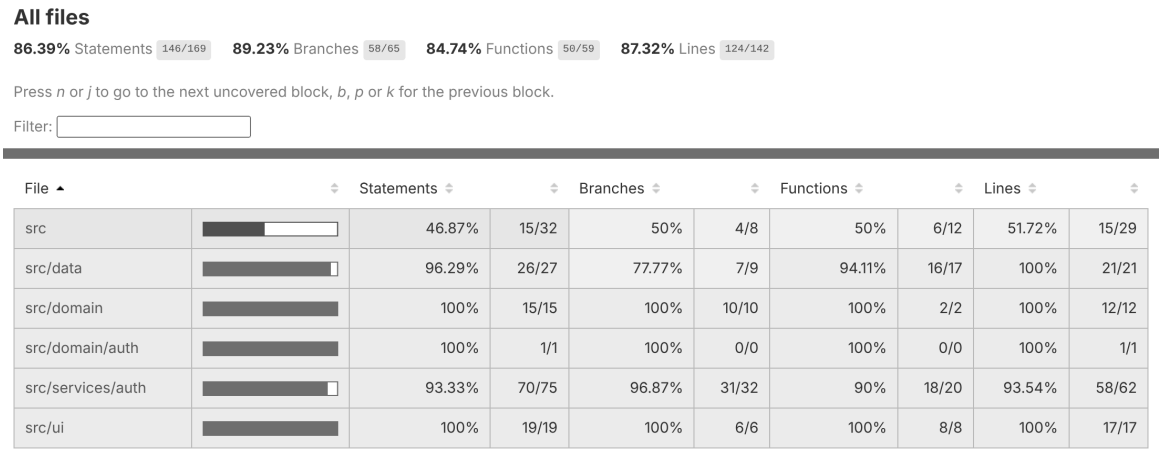
  if (day <= lastActiveDay) return 'active';
  if (day <= lastGraceDay) return 'grace';
  return 'expired';
}
```

```
PASS src/domain/membership.test.ts
Tests:       9 passed, 9 total
```

The tests stayed green, so the refactor changed no behavior. TypeScript types are omitted above for space; the full file is in the repository.

Coverage Report and Gap Analysis

Generated with `npm run test:coverage` (Jest with Istanbul) on October 23, 2026. Type-only files with no runnable code, the test helper file, and the old before/after design-pattern examples are excluded in package.json, so they don't distort the numbers.



File	Statements	Branches	Lines
domain/membership.ts	100%	100%	100%
services/auth/AuthSession.ts	100%	100%	100%
services/auth/LocalAuthProvider.ts, DemoAuthProvider.ts, LoginValidator.ts	100%	100%	100%
ui/ProtectedRoute.tsx, ui/SignUpPage.tsx, Login.tsx	100%	100%	100%
data/LocalStorageRepository.ts	100%	80%	100%
services/auth/SignUpService.ts	95.7%	93.8%	100%
data/LocalStorageUserRepository.ts	88.9%	75%	100%
App.js	0%	0%	0%
services/auth/PasswordHasher.ts	0%	100%	0%
All files	**86.4%**	**89.2%**	**87.3%**

What Is Untested, and Why

- **App.js (0%).** This file only wires the real classes together, saves the demo accounts and lists the routes. Rendering it in Jest would mean real localStorage and real password hashing, which Jest doesn't have. Its job is checked by system and acceptance testing instead: every step of the demo script runs through it. The pieces it connects (routes, guard, sign-up form, session) are each covered by unit and integration tests.
- **PasswordHasher.ts (0%).** It calls the browser's crypto.subtle, which doesn't exist in Jest's simulated browser. We confirmed this with a one-line test that found crypto.subtle undefined. Tests use fakeHasher instead. The real hasher runs in every acceptance test, since logging in as staff@demo.gym only works if hashing is correct.
- **Default storage arguments (partial branches).** The uncovered branches in LocalStorageRepository.ts and LocalStorageUserRepository.ts are the default value "use the real localStorage when none is given". Tests always pass a fake on purpose, so they never touch the real browser storage; the app itself always uses the default.
- **SignUpService line 52.** The safety message for "account created but sign-in failed" can only happen if the app is wired with a session that uses a different user list than the sign-up service. That's a set-up mistake rather than something a user can cause, so it's left untested.

Next Steps

- Test PasswordHasher with Node's own Web Crypto (from the crypto module), so the real SHA-256 code is covered too.
- Add a small render test for App.js with storage and hashing passed in, once the app's startup code is moved into a function that accepts them.
- Keep the 70% rule as a gate in every pull request: run npm run test:coverage and check the changed files before approving.